F25-385-D-VOICE

Project Team

Shaheer Zaman 22I-0805
Najam Hassan 22I-1332
Aliyan Zafar 20I-2414

Session 2022-2026

Supervised by

Mr. Almas Khan

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

Oct, 2025

Contents

1 Introduction **1**
1.1 Problem Statement . . . 1
1.2 Scope . . . 1
1.3 Modules . . . 2
1.3.1 Module 1: User Management and Data Collection . . . 2
1.3.2 Module 2: Machine Learning and Real-Time Inference . . . 2
1.3.3 Module 3: User Interface and Visualization . . . 3
1.3.4 Module 4: Deployment and Maintenance . . . 3
1.4 User Classes and Characteristics . . . 3

2 Project Requirements **5**
2.1 Use-case/Event Response Table/Storyboarding . . . 5
2.2 Functional Requirements . . . 5
2.2.1 Module 1: User Management and Data Collection . . . 5
2.2.2 Module 2: Machine Learning and Real-time Inference . . . 6
2.2.3 Module 3: User Interface and Visualization . . . 6
2.2.4 Module 4: Deployment and Maintenance . . . 6
2.3 Non-Functional Requirements . . . 7
2.4 Domain Model . . . 7

3 System Overview **9**
3.1 Architectural Design . . . 9
3.1.1 Module Relationships and System Decomposition . . . 9
3.1.2 Architectural Patterns . . . 10
3.2 Design Models . . . 11
3.2.1 Activity Diagram: End-to-End Translation Process . . . 11
3.2.2 Class Diagram: Object-Oriented Component Structure . . . 12
3.2.3 Class-Level Sequence Diagram . . . 13
3.2.4 State Transition Diagram . . . 13
3.3 Design Models for Procedural Approach . . . 18
3.3.1 Data Flow Diagram Level 0 (Context Diagram) . . . 18
3.3.2 Data Flow Diagram Level 1 (Detailed Process Decomposition) . . . 19
3.3.3 System-Level Sequence Diagram . . . 19

3.4 Data Design 24

List of Figures

2.1 domain model 8

3.1 Layered Architectural Diagram 11
3.2 Activity Diagram 15
3.3 Class Diagram 16
3.4 Class-level Sequence Diagram 17
3.5 State Transition Diagram 18
3.6 Data Flow Diagram level 0 21
3.7 Data Flow Diagram level 1 22
3.8 System-Level Sequence Diagram 23

List of Tables

2.1 Functional Requirements for Module 1: User Management and Data Collection 5
2.2 Functional Requirements for Module 2: Machine Learning and Real-time Inference 6
2.3 Functional Requirements for Module 3: User Interface and Visualization . 6
2.4 Functional Requirements for Module 4: Deployment and Maintenance . . 6
2.5 Combined Non-Functional Requirements 7

3.1 Data Design of VOICE PSL System 24

Chapter 1

Introduction

Over 70 million deaf or hard-of-hearing individuals worldwide face daily communication barriers. In Pakistan, communication frequently relies on Pakistani Sign Language (PSL), yet most hearing individuals do not understand PSL. Existing tools are limited to alphabet recognition, lack real-time performance, are not web-accessible, or are pro- hibitively expensive. Word-level PSL recognition is underexplored, and there is no widely available, real-time, browser-based system that supports bidirectional communication (sign → text/speech and text/speech → sign via avatar).

1.1 Problem Statement

Design and implement a real-time, web-accessible system that (i) recognizes dynamic, word-level PSL gestures and converts them to readable text and synthesized speech (Urdu/English), and (ii) supports reverse communication by animating a PSL avatar from input text/speech, thereby enabling inclusive, two-way interaction between deaf/mute and hearing users.

1.2 Scope

In-Scope

- Word-level PSL (selected vocabulary) with dynamic gesture recognition.
- Browser-based live capture (WebRTC/getUserMedia), on-device preprocessing, and model inference.
- Urdu/English text and speech output; confidence prompts.
- Avatar animation of curated PSL words (stepwise playback).

- Learning mode: practice, verification, and feedback.
- Evaluation: accuracy, latency (FPS), robustness across users and environments.

Out-of-Scope (for this phase)

- Full PSL sentence-level translation and continuous sign language parsing.
- Domain-specific language modeling or context-aware dialogue systems.
- Native mobile apps (a responsive PWA is sufficient for this phase).

1.3 Modules

Write down the modules of the proposed project. Each module should highlight features, using bulleted/numbered notation. When developing both a mobile app and a web app, group the modules according to the system types, such as, Client Web App, Client Mobile App, Admin Web App etc.

1.3.1 Module 1: User Management and Data Collection

This module allows users (signers, administrators, and testers) to manage their access, collect PSL gesture data, and maintain metadata for training and testing.

1. User registration and authentication
2. Webcam-based PSL gesture recording
3. Automatic MediaPipe hand landmark extraction
4. Data labeling and secure storage

1.3.2 Module 2: Machine Learning and Real-Time Inference

This module focuses on model training and real-time recognition of PSL gestures through browser-based inference using TensorFlow.js.

1. CNN+LSTM-based model training for PSL gestures
2. Model quantization and optimization for browser inference

3. Real-time FPS and latency monitoring
4. Dynamic text/speech output based on recognition results

1.3.3 Module 3: User Interface and Visualization

This module manages the front-end of the application, providing users with an intuitive interface and visualization of both PSL-to-text and text-to-PSL translation.

1. Responsive bilingual (Urdu/English) interface
2. 3D avatar playback for text-to-sign mode
3. Interactive learning module for practice and feedback
4. System analytics and performance dashboards

1.3.4 Module 4: Deployment and Maintenance

This module ensures the continuous deployment, system reliability, and integration of user feedback.

1. CI/CD pipeline and cloud deployment
2. Cross-platform browser testing
3. Error reporting and model update handling
4. Logging and system monitoring

1.4 User Classes and Characteristics

User Classes and Characteristics

User class	Description
Deaf/Mute Users	Primary users who will communicate via PSL gestures. They require a simple UI with live camera access and feedback.
Hearing Users	Family members, colleagues, or service providers who communicate with PSL users using text or voice input
Educators and NGOs	Institutions (e.g., Deaf Reach, FESF) who may use the learning module to train students or create new datasets.
Researchers	Users who may reuse the data pipeline or trained models for further AI and accessibility research.
Administrators	Responsible for system maintenance, retraining models, and managing application performance logs.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

The VOICE PSL system is an interactive, real-time web application that enables translation between Pakistani Sign Language (PSL), text, and voice. Therefore, a use-case-based approach is used to represent functional requirements. The following tables summarize the detailed requirements gathered for this system.

2.2 Functional Requirements

2.2.1 Module 1: User Management and Data Collection

ID	Iteration	Requirement Description	Priority	Acceptance Criteria
FR-01	Iteration 1	User registration and login functionality	High	Users can register, log in, and log out successfully
FR-02	Iteration 1	Dataset collection and recording interface for PSL gestures	High	System records gestures from webcam with consent tracking
FR-03	Iteration 1	MediaPipe-based hand landmark detection	High	Landmarks detected in 95% of frames
FR-04	Iteration 1	Data labeling and metadata storage for each recording	Medium	All samples stored with signer ID and metadata
FR-05	Iteration 1	Data preprocessing and validation pipeline	Medium	Processed dataset passes data integrity tests

Table 2.1: Functional Requirements for Module 1: User Management and Data Collection

2.2.2 Module 2: Machine Learning and Real-time Inference

ID	Iteration	Requirement Description	Priority	Acceptance Criteria
FR-06	Iteration 2	Deep learning model training (CNN+LSTM) for PSL recognition	High	Model achieves 80% Top-1 accuracy on validation set
FR-07	Iteration 2	Real-time inference engine using TensorFlow.js	High	Inference latency 50ms during runtime
FR-08	Iteration 2	Voice/Text output for recognized PSL gestures	High	Predicted signs converted to text/voice within 1s
FR-09	Iteration 2	Confidence and Top-k predictions display	Medium	UI shows top-3 predictions with confidence levels
FR-10	Iteration 2	Performance visualization dashboard	Medium	Dashboard shows metrics (accuracy, FPS, latency)

Table 2.2: Functional Requirements for Module 2: Machine Learning and Real-time Inference

2.2.3 Module 3: User Interface and Visualization

ID	Iteration	Requirement Description	Priority	Acceptance Criteria
FR-11	Iteration 3	3D avatar playback for text-to-sign translation	High	Avatar reproduces PSL gestures accurately
FR-12	Iteration 3	Interactive learning and practice module	Medium	System gives feedback and similarity score to user signs
FR-13	Iteration 3	Multi-language UI (Urdu/English toggle)	Medium	UI fully switches between Urdu and English
FR-14	Iteration 3	System performance and analytics dashboard	Low	Admin can view performance reports and charts

Table 2.3: Functional Requirements for Module 3: User Interface and Visualization

2.2.4 Module 4: Deployment and Maintenance

ID	Iteration	Requirement Description	Priority	Acceptance Criteria
FR-15	Iteration 4	Model retraining and update integration	Medium	Retrained model deployed with no downtime
FR-16	Iteration 4	Cross-platform testing (desktop/mobile)	Medium	Application works on Chrome, Edge, and Android
FR-17	Iteration 4	User feedback and issue reporting system	Low	Users can submit feedback directly from the web app
FR-18	Iteration 4	Final deployment and demo preparation	High	Production version deployed and presented successfully

Table 2.4: Functional Requirements for Module 4: Deployment and Maintenance

2.3 Non-Functional Requirements

ID	Requirement Description	Priority	Acceptance Criteria
NFR-01	Real-time performance: End-to-end latency 50ms	High	Verified on 80% of devices during testing
NFR-02	Accuracy: Minimum 80% Top-1 and 90% Top-3 recognition accuracy	High	Model evaluation meets accuracy thresholds
NFR-03	Privacy: Store only landmark data, no raw videos	High	Data audit confirms anonymization
NFR-04	Security: Use HTTPS/TLS for all communications and encryption at rest	High	Penetration testing shows no high-severity vulnerabilities
NFR-05	Availability: Maintain 99% uptime for production system	Medium	Monitored logs confirm less than 1% downtime
NFR-06	Scalability: Support 500+ concurrent users without performance drop	Medium	Load testing verifies constant latency under peak load
NFR-07	Maintainability: Modular, documented code with 60% unit test coverage	Medium	Code review and CI validation successful
NFR-08	Usability: WCAG AA compliant interface for accessibility	High	User testing with hearing-impaired participants successful
NFR-09	Portability: Support for desktop browsers and Android devices	Medium	Cross-device tests pass with no major issues
NFR-10	Localization: Full Urdu and English language support	Medium	UI verified for both languages across all modules

Table 2.5: Combined Non-Functional Requirements

2.4 Domain Model

The domain model for the VOICE PSL system represents entities such as **User**, **Gesture**, **Model**, **Avatar**, **Translation**, and **Feedback**. These entities interact to handle data capture, machine learning inference, result generation, and visualization.

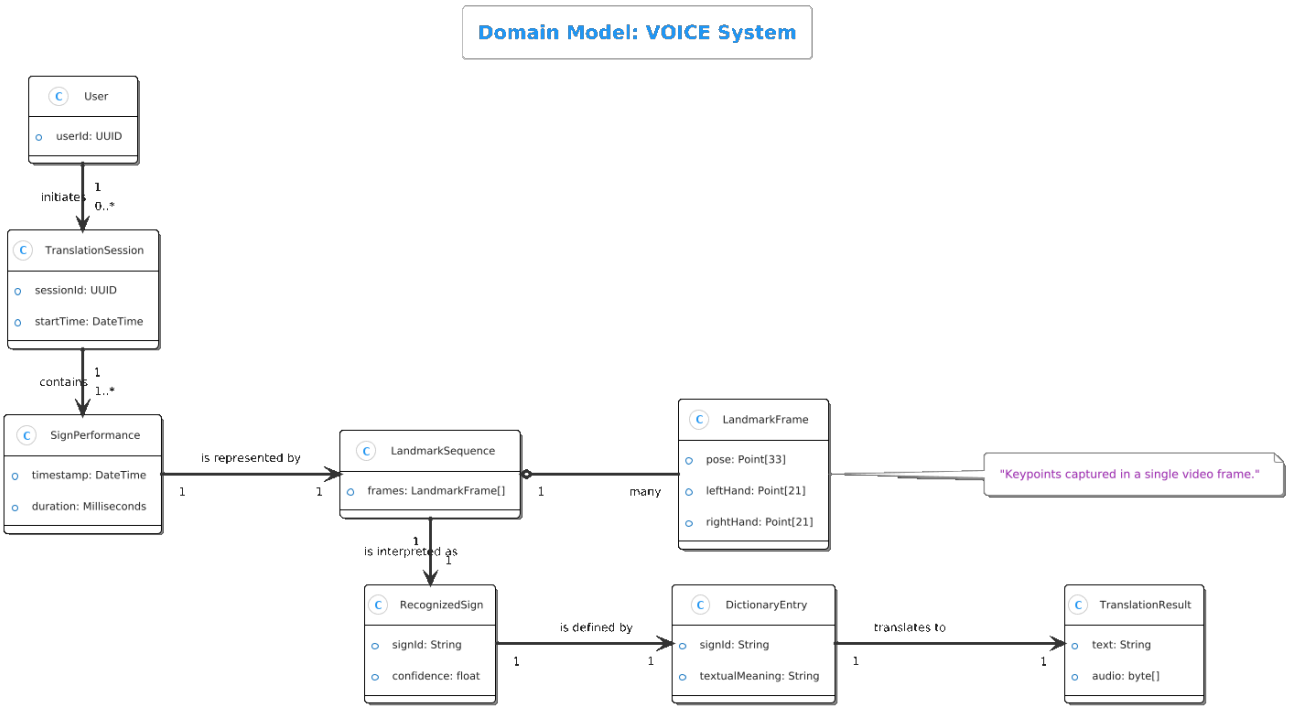

Figure 2.1: domain model

Chapter 3

System Overview

Give a general description of the functionality, context, and design of your project. Provide any background information if necessary.

3.1 Architectural Design

The VOICE PSL system employs a **Layered Architecture** pattern to separate concerns and ensure modularity across the application. This architecture decomposes the system into four distinct layers: the Presentation Layer (client-side), the Application Layer (back-end orchestration), the Domain/Service Layer (business logic), and the Inference & Data Layer (machine learning and storage).

3.1.1 Module Relationships and System Decomposition

The system is structured around four primary modules that work in tandem to achieve real-time bidirectional translation between Pakistani Sign Language, text, and synthesized speech:

1. **Module 1: User Management and Data Collection** handles user authentication, webcam-based gesture recording, and automatic extraction of hand landmarks using MediaPipe. This module feeds raw data into the system pipeline.

2. **Module 2: Machine Learning and Real-Time Inference** processes landmark sequences through a CNN+LSTM deep learning model to recognize PSL gestures and generate predictions with confidence scores. This module operates on quantized TensorFlow.js models for browser-based inference achieving sub-50ms latency.

3. **Module 3: User Interface and Visualization** provides a responsive bilingual (Urdu/English) interface, displays recognition results with confidence levels, and renders 3D avatar animations for text-to-sign translation. This layer also includes interactive learning and practice modules.

4. **Module 4: Deployment and Maintenance** ensures continuous deployment through CI/CD pipelines, cross-platform browser compatibility, and system monitoring for reliability.

These modules communicate through well-defined interfaces: the client sends video frames via WebSocket to the server, which processes them through the recognition pipeline and returns JSON payloads containing predicted sign IDs and audio data. The UI consumes this output to display translations and trigger text-to-speech synthesis.

3.1.2 Architectural Patterns

The system combines a **Client-Server architecture** at the network level with a **Layered (N-Tier) architecture** at the application level:

- **Presentation Layer:** Browser-based UI (HTML/CSS/React), Camera Media API for video capture, WebSocket client for bidirectional communication

- **Application Layer (Python-Flask):** API endpoints, WebSocket server, session management, and Translation Orchestrator that routes data flow between services

- **Domain/Service Layer:** Sign Recognition Service (landmark extraction via MediaPipe and model inference), Text-to-Speech Service (TTS synthesis for Urdu/English output)

- **Inference & Data Layer:** TensorFlow/PyTorch inference engine, trained CNN+LSTM model weights, Sign Dictionary (JSON/DB lookup), and data storage (Supabase/Firebase for user profiles, Google Cloud Storage for gesture datasets)

Communication flows vertically through secure WebSocket (WSS) connections, with the server managing frame buffering, landmark extraction, and inference orchestration. This separation allows independent scaling of each layer and supports the real-time performance requirements (50ms end-to-end latency).

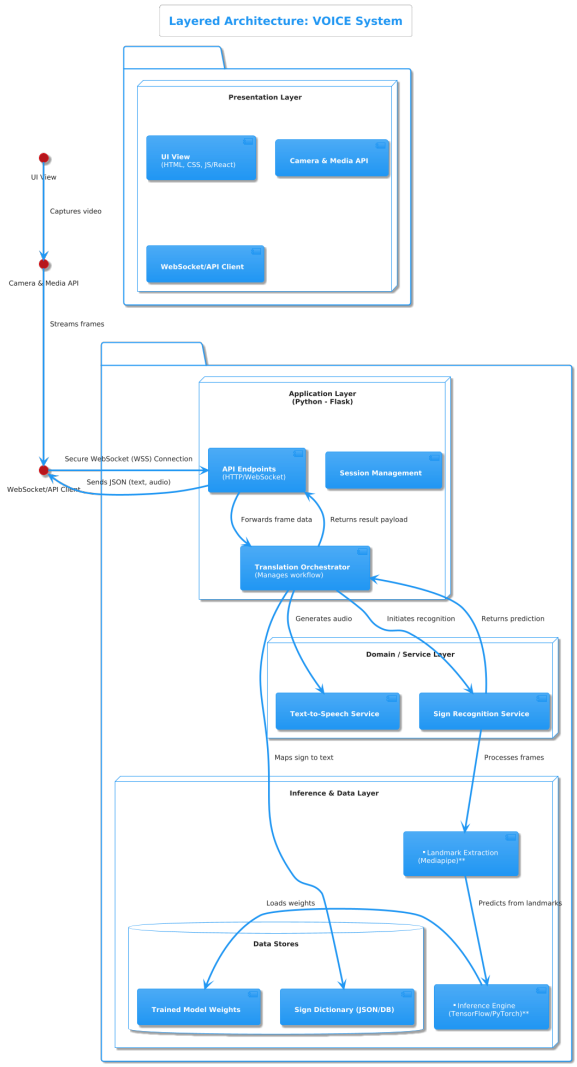

Figure 3.1: Layered Architectural Diagram

3.2 Design Models

The VOICE PSL system employs a hybrid design approach combining object-oriented and procedural elements. The following design models represent the system's dynamic behavior, component interactions, and state management.

3.2.1 Activity Diagram: End-to-End Translation Process

The Activity Diagram illustrates the complete workflow from user gesture input through text/speech output. Key control flows include:

- **Initialization Phase:** User navigates to the VOICE URL, grants camera permission, and initializes WebSocket connection

- **Capture Phase:** Camera feed is activated; frames are continuously streamed to the server for processing
- **Processing Phase:** Server receives frames, extracts hand landmarks using MediaPipe, and appends to a temporal buffer
- **Inference Phase:** When the buffer reaches a minimum sequence length (e.g., 30 frames), landmarks are fed to the CNN+LSTM recognition model
- **Output Phase:** If confidence exceeds a threshold, the recognized sign is mapped to text and synthesized to speech via TTS. Low-confidence predictions are discarded with user notification
- **Feedback Loop:** Buffer is cleared for the next sign, and the cycle repeats or terminates based on user action

Decision points include permission denial (error handling), confidence threshold validation, and user intention to continue or exit the session.

3.2.2 Class Diagram: Object-Oriented Component Structure

The Class Diagram represents the primary classes and their relationships:

- **Client-side (JavaScript):**
  - `UIManager`: Manages UI rendering and result display
  - `CameraController`: Handles webcam access and frame capture via MediaStream API
  - `WebSocketHandler`: Manages bidirectional WebSocket communication with the server
- **Server-side (Python):**
  - `FlaskServer`: HTTP and WebSocket endpoint handler
  - `TranslationService`: Orchestrates the translation workflow
  - `LandmarkExtractor`: Wraps MediaPipe holistic landmark detection
  - `SignPredictor`: Encapsulates the trained TensorFlow model and dictionary lookup

The UIManager uses CameraController to acquire frames; CameraController relays frames through WebSocketHandler to the server's FlaskServer. FlaskServer invokes TranslationService, which coordinates LandmarkExtractor and SignPredictor to produce predictions. Results flow back to UIManager for rendering.

3.2.3 Class-Level Sequence Diagram

This diagram captures the interaction sequence between UIManager, CameraController, WebSocketHandler, and the server-side TranslationService during a single sign recognition cycle:

1. UIManager initializes CameraController with a frame callback
2. CameraController captures frames and passes each to WebSocketHandler
3. WebSocketHandler buffers frames and sends them to FlaskServer
4. FlaskServer forwards the frame to TranslationService
5. TranslationService calls LandmarkExtractor.extract(image) returning hand keypoints
6. TranslationService appends landmarks to a temporal sequence
7. Upon sequence completion, TranslationService invokes SignPredictor.predictSequence()
8. SignPredictor returns a dictionary with sign ID and confidence
9. TranslationService maps the sign ID to text and sends result back to UIManager
10. UIManager displays text and triggers TTS playback

3.2.4 State Transition Diagram

The system operates through well-defined states:

- **Idle:** Initial state; awaiting user action
- **Capture:** Camera is active and frames are being recorded
- **Processing:** Landmarks are being extracted and buffered
- **Inference:** The recognition model is processing a complete sequence
- **Output:** Results are being displayed and audio is playing
- **Feedback:** System awaits user confirmation or reset
- **Error:** System encountered a failure (camera error, extraction failure, model error, or output failure)

Transitions occur based on events: `Start Capture` → Capture, `Frames Recorded` → Processing, `Landmarks Extracted` → Inference, `Prediction Complete` → Output, `Display Complete / Reset` → Idle. Error conditions from any state transition to Error; from Error, `System Failure` loops to Idle or the Error state persists depending on recovery logic.

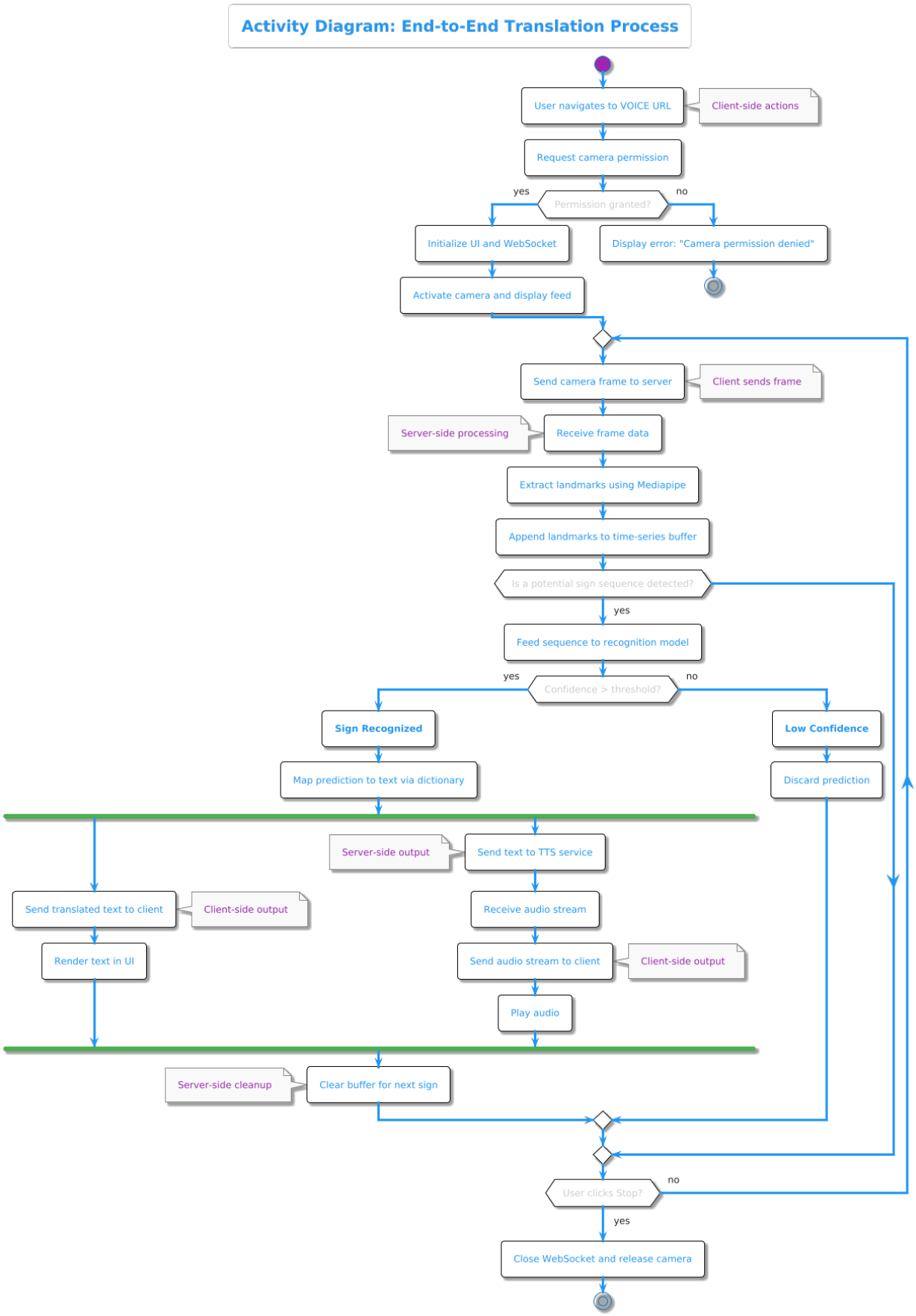

Figure 3.2: Activity Diagram

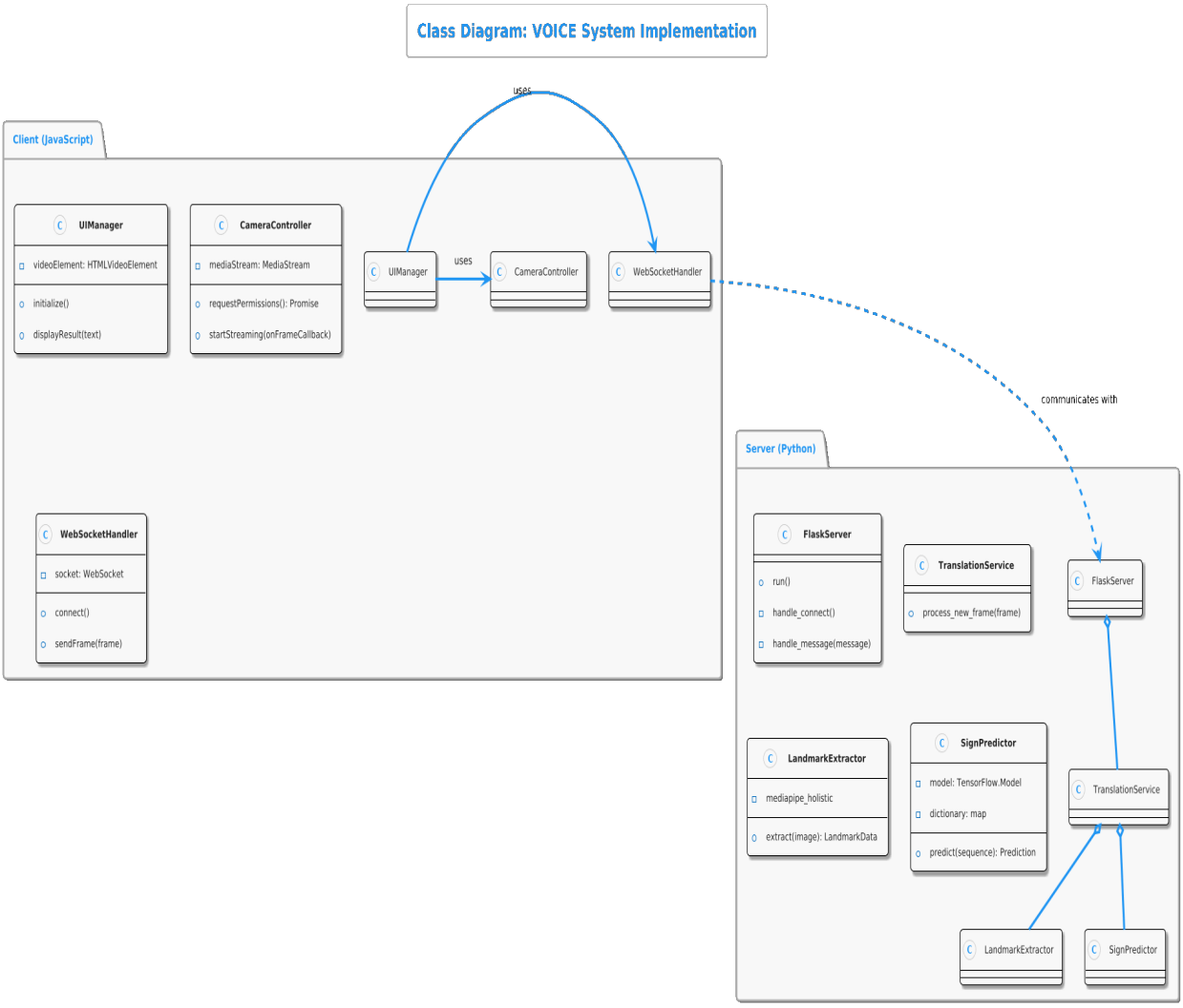

Figure 3.3: Class Diagram

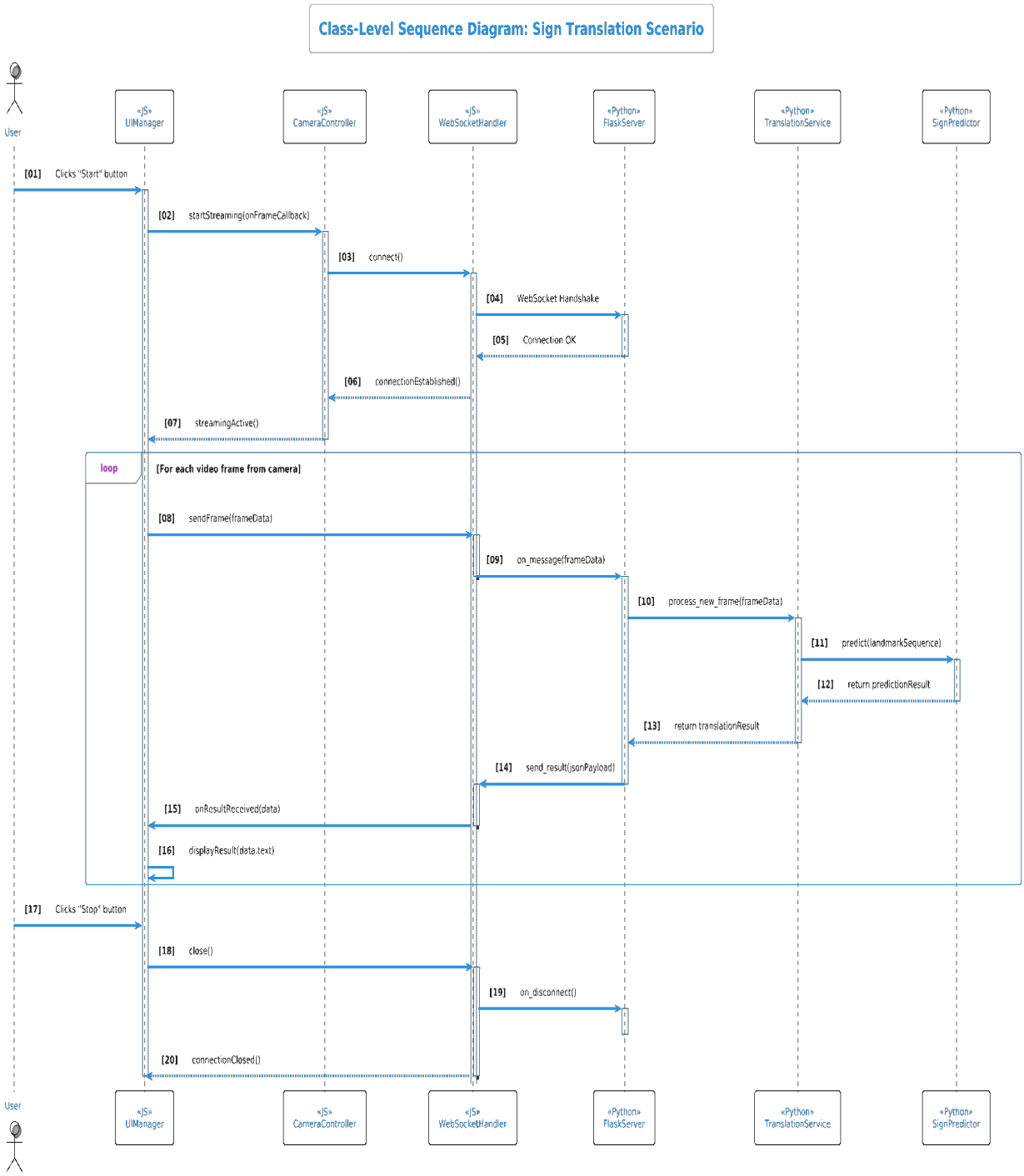

Figure 3.4: Class-level Sequence Diagram

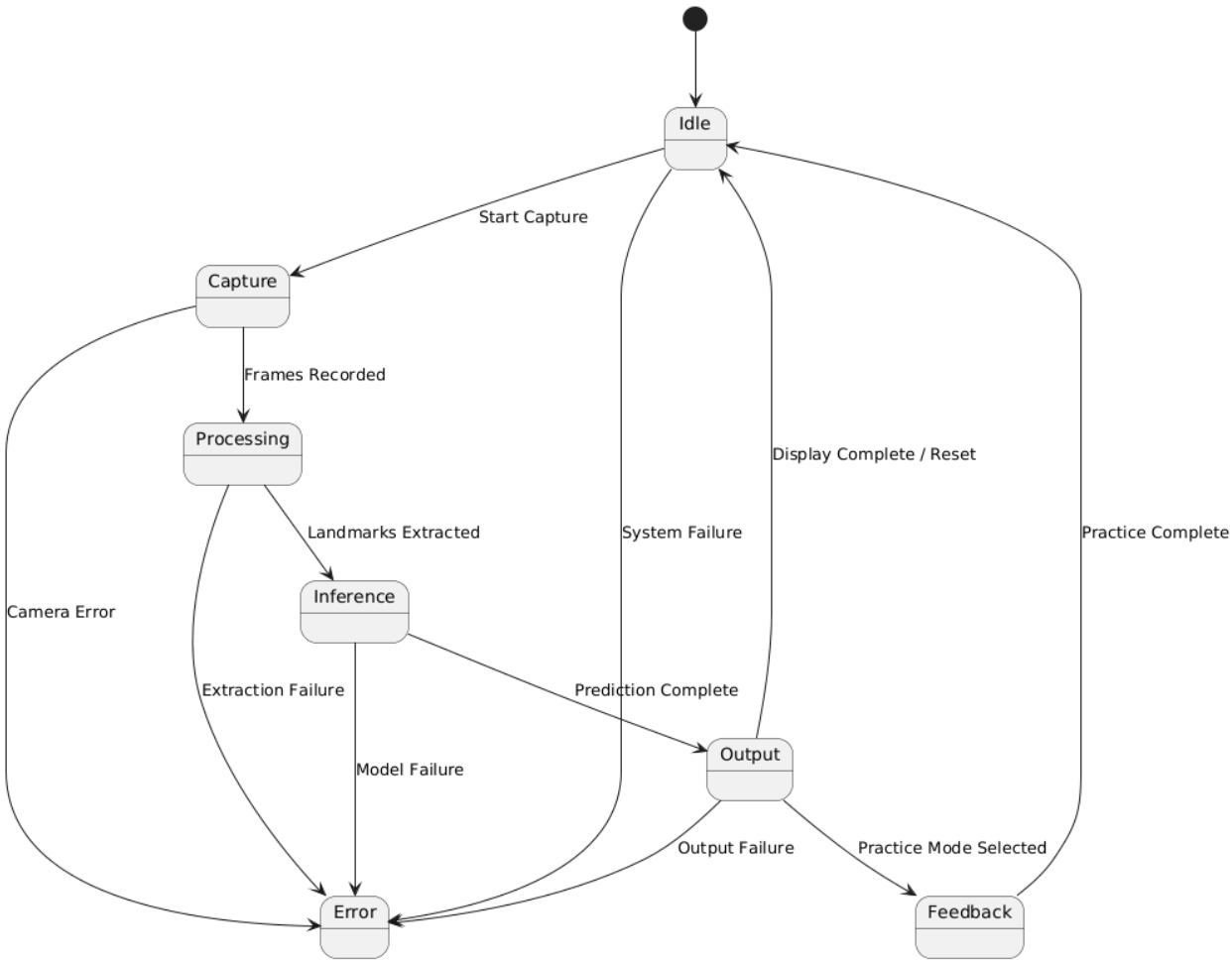

Figure 3.5: State Transition Diagram

3.3 Design Models for Procedural Approach

While the architecture is primarily object-oriented, the data processing pipeline employs procedural elements. The following models detail this procedural flow.

3.3.1 Data Flow Diagram Level 0 (Context Diagram)

The Level 0 DFD presents the system as a single process with external entities:

- **User:** Provides gesture input (video frames) and receives text/audio/avatar animation output
- **VOICE PSL Translator:** Single-bubble process encompassing all transformation
- **Output Devices:** Display and audio output

- **Dataset Store:** Persistent storage for gesture samples and model weights

Data stores: gesture input flows in, recognized text and audio flows out, feedback logs and training data are archived.

3.3.2 Data Flow Diagram Level 1 (Detailed Process Decomposition)

Level 1 expands the single process into five sub-processes:

1. **1.1 Capture Gesture Frames:** User provides video via webcam; frames are extracted as a sequence
2. **1.2 Landmark Extraction:** Frame sequences pass through MediaPipe; output is normalized hand landmark coordinates (21 keypoints per hand per frame)
3. **1.3 Preprocessing:** Landmarks are normalized for scale and position invariance
4. **1.4 Model Inference:** Preprocessed sequences feed into the CNN+LSTM model; output is a predicted sign label with confidence score
5. **1.5 Output Generation:** Predicted label triggers text-to-speech synthesis (via TTS service) and avatar animation playback (from dictionary); feedback and logs are stored

Data flows: Frame Sequence → Landmark Extraction → Hand Landmarks → Preprocessing → Normalized Data → Model Inference → (Predicted PSL Label, Retraining Data) → Output Generation → (Feedback Data, Text + Audio + Avatar Animation)

Data stores: D1 (PSL Dataset), D2 (User Feedback Logs), D3 (Model Checkpoints).

3.3.3 System-Level Sequence Diagram

This diagram illustrates the temporal orchestration across five key system components during a complete translation cycle:

01 User clicks Start on Browser UI

02 Browser UI establishes WebSocket connection to Server

03 Server sends WebSocket Handshake

04 Server confirms Connection OK

05 Client registers connection established

06 Client activates camera via Camera API

07 User sees live camera feed

08–09 For each video frame: Browser sends frame data, Server streams frame

10 Recognition Service processes frame and extracts landmarks

11 Recognition Service returns predicted sign and confidence

12 Server generates audio prediction via TTS Service

13 TTS Service returns audio data

14 Server sends result (text, audio) to Browser UI

15 Browser receives result payload

16 Browser displays recognized text

17 Browser plays synthesized audio

18 Server clears buffer for next sign

The loop continues until the user stops the session, triggering WebSocket closure and camera release. The diagram emphasizes asynchronous message passing and service orchestration.

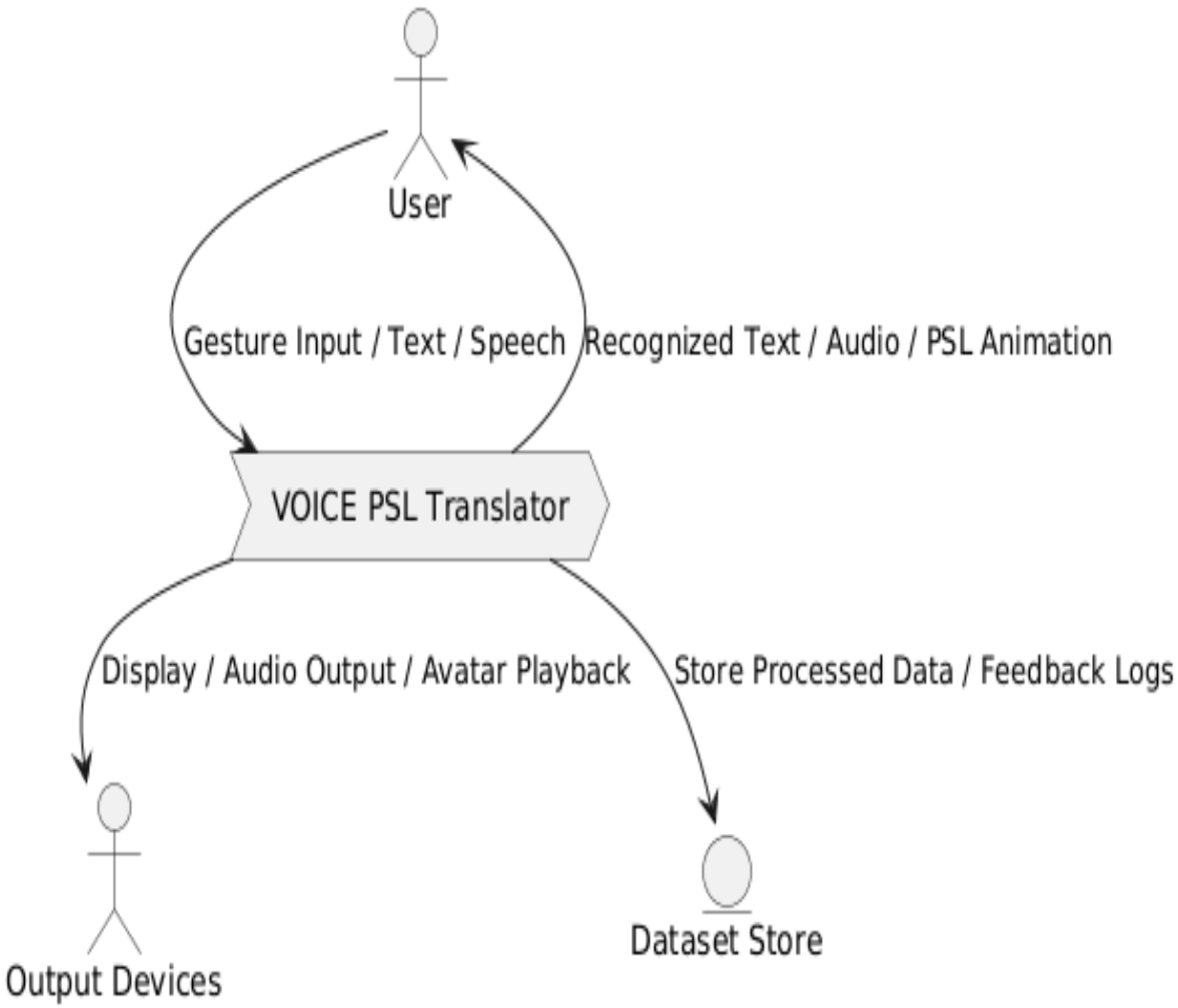

Figure 3.6: Data Flow Diagram level 0

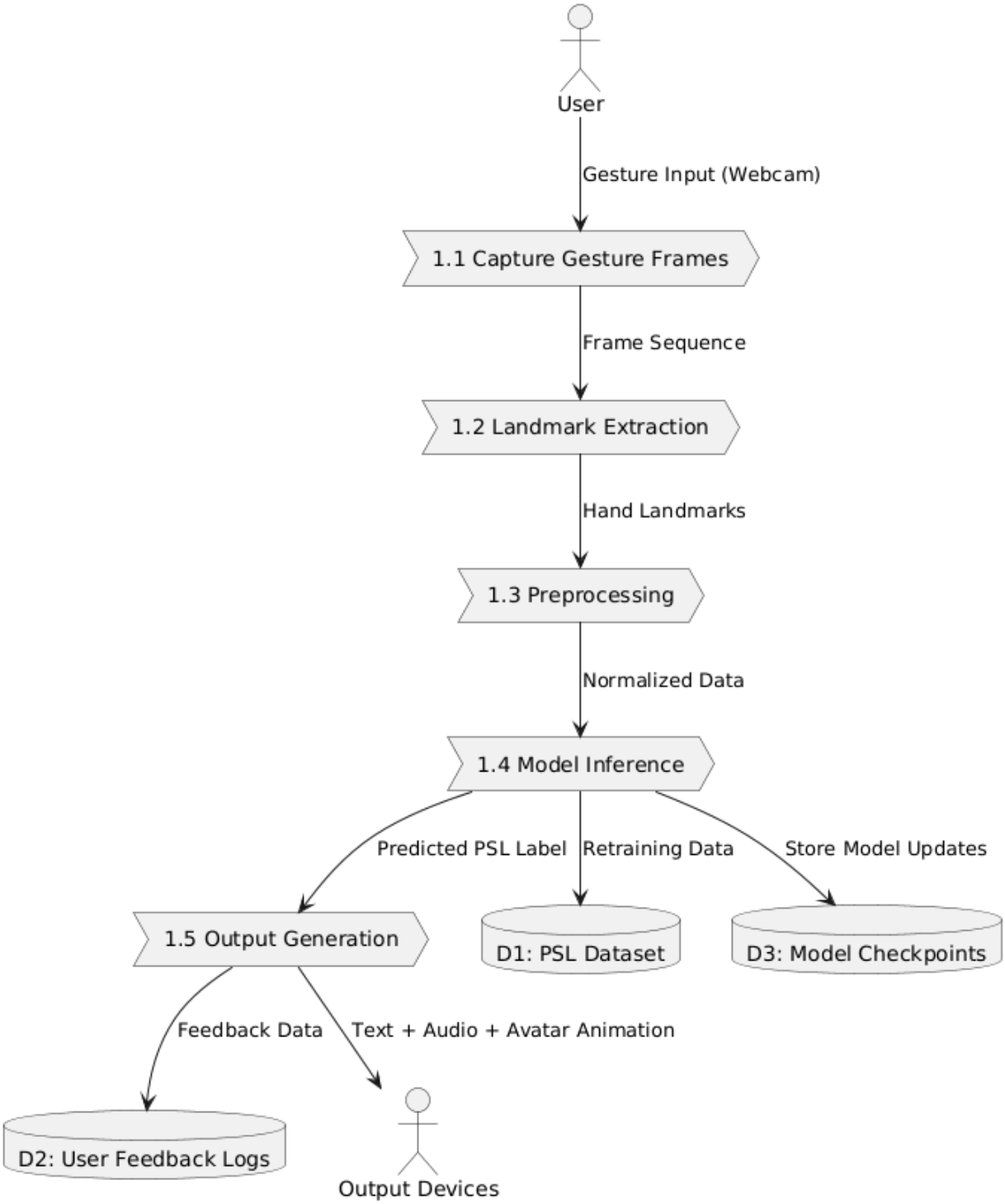

Figure 3.7: Data Flow Diagram level 1

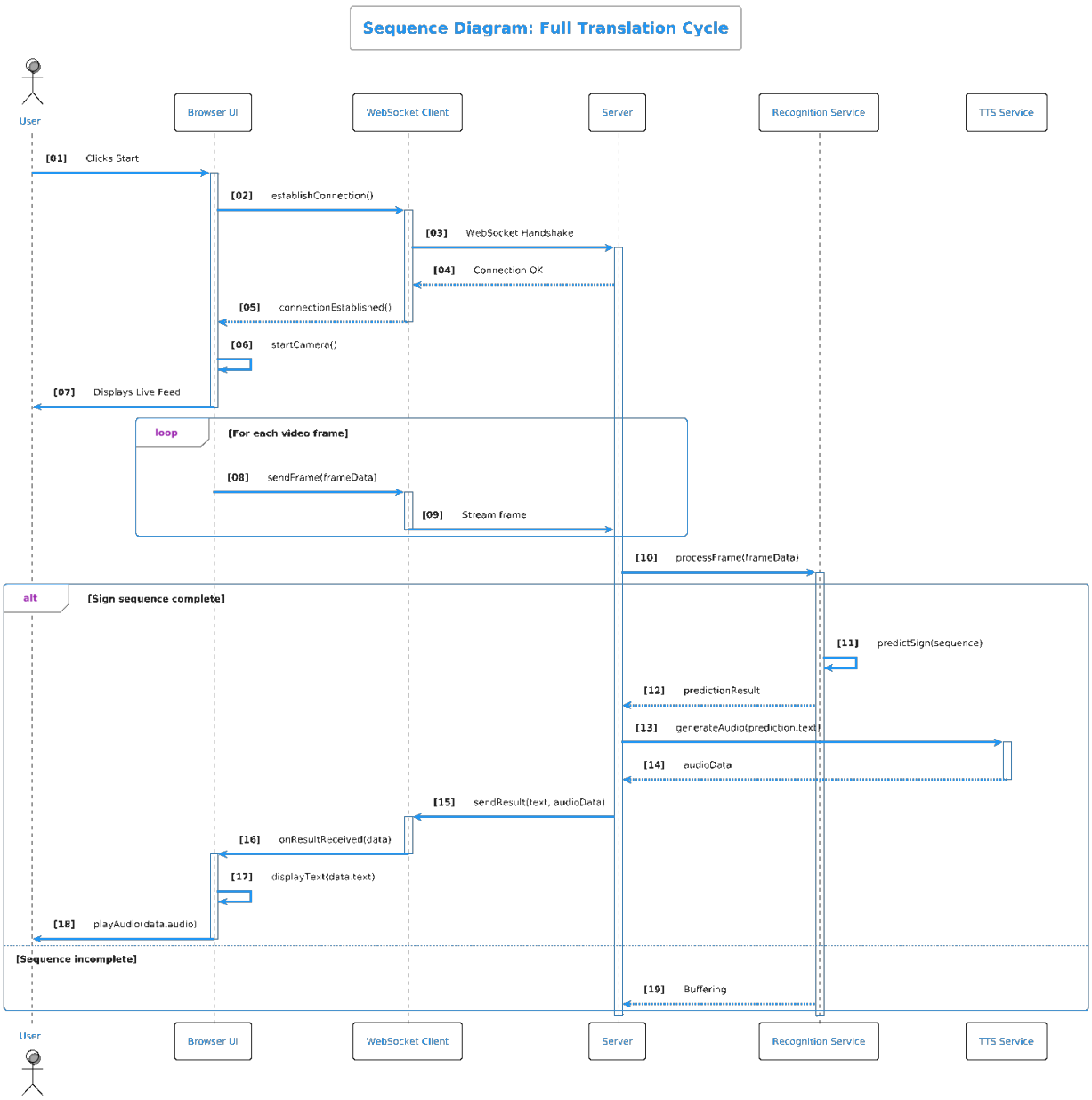

Figure 3.8: System-Level Sequence Diagram

3.4 Data Design

The data design focuses on how information is stored, accessed, and processed across the VOICE PSL system.

Table 3.1: Data Design of VOICE PSL System

Data Entity	**Description**	**Storage Method**
User Profile	Stores user credentials and preferences	Supabase / Firebase Auth
Gesture Dataset	Raw videos and extracted landmark coordinates	Cloud storage (Google Drive / Supabase bucket)
Model Weights	Pretrained deep learning models (CNN+LSTM)	Server-side / ONNX model file
Inference Data	Temporary storage for current session landmarks	Browser memory / IndexedDB
Feedback Logs	Stores user performance metrics and similarity scores	Supabase DB table
Avatar Mapping	Keyframes and animation scripts for PSL gestures	JSON-based 3D motion data